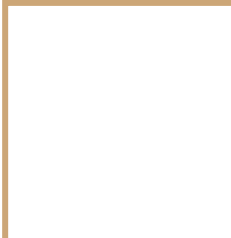


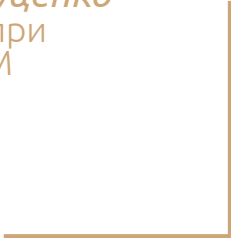
LLM IN & OUT: TWENTY MINUTE ADVENTURE.





LLM IN & OUT: TWENTY MINUTE ADVENTURE. AS IF...

В какие ловушки *Иван Луценко*
попадал и попадает при
использовании LLM



ОБО МНЕ

Роль: android техлид в Bereke Business



ОБО МНЕ

Роль: android техлид в Bereke Business

Специализация: платформенная
android разработка



ОБО МНЕ

Роль: android техлид в Bereke Business

Специализация: платформенная
android разработка

Опыт: петы, outsource проекты,
банковское приложение



ОБО МНЕ

Роль: android техлид в Bereke Business

Специализация: платформенная
android разработка

Опыт: петы, outsource проекты,
банковское приложение

Текущий фокус: AI-ассистированная
разработка и управление проектами

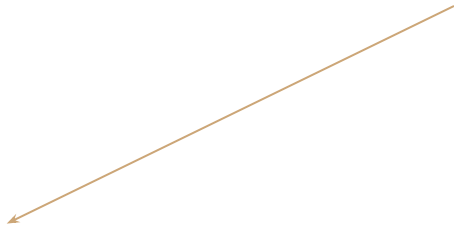


LLM В ТЕОРИИ

Написал что-то в чатик

LLM В ТЕОРИИ

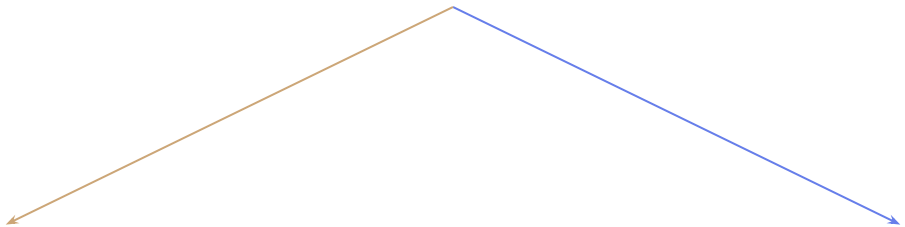
Написал что-то в чатик



- угадывание контекста
- точный и детальный ответ
- идеальное, готовое к работе решение

LLM В ТЕОРИИ VS LLM НА ПРАКТИКЕ

Написал что-то в чатик

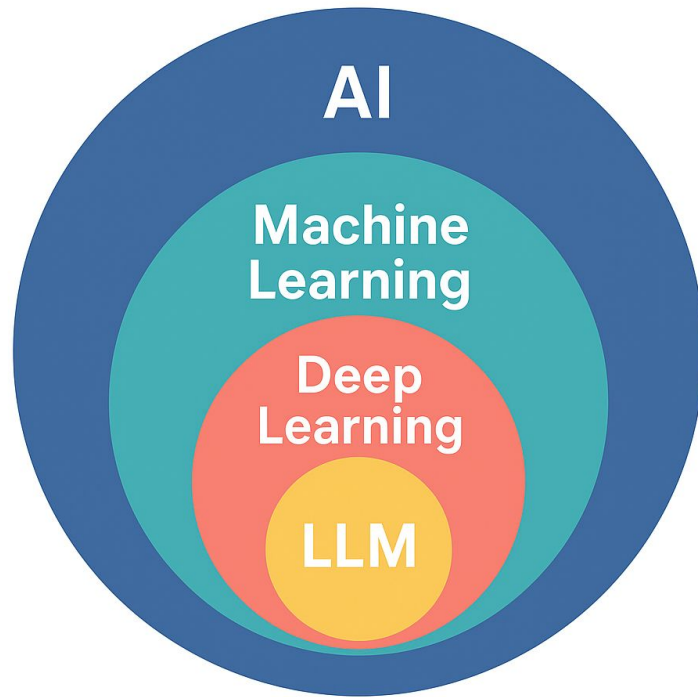


- угадывание контекста
- точный и детальный ответ
- идеальное, готовое к работе решение

- додумывание контекста
- примерный набросок структуры
- черновик, требующий разбора и доработки

Ловушка #1: "LLM это ИИ, сам разберётся"

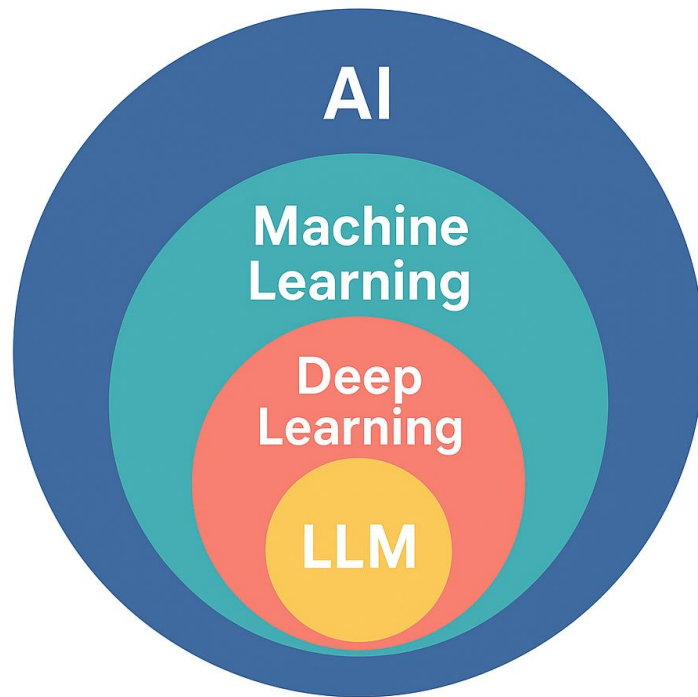
Переоценка автономности.



Ловушка #1: "LLM это ИИ, сам разберётся"

Переоценка автономности.

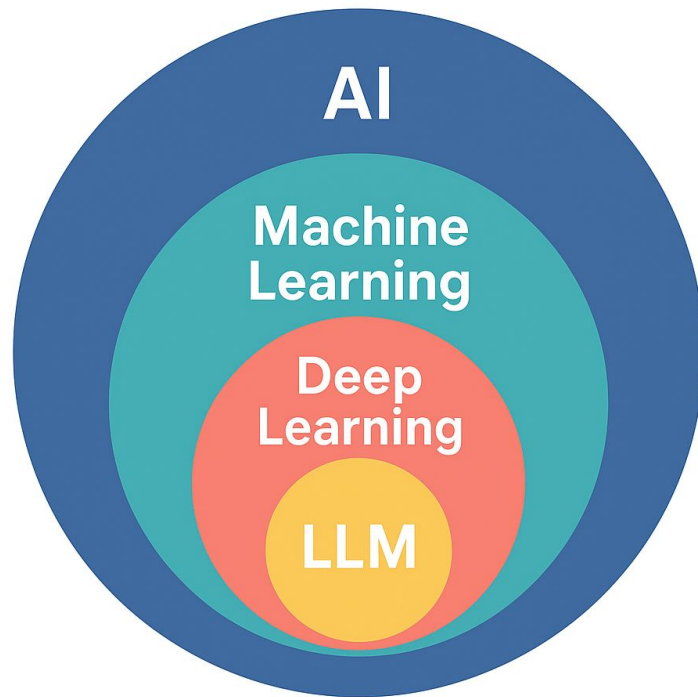
- Забываешь, что LLM — это инструмент
- Получаешь бесполезный ответ
- "Пробовал я эти ваши LLM..."



Ловушка #1: "LLM это ИИ, сам разберётся"

Совет:

Чёткий контекст + конкретные
ожидания + проверка результата



PasswordValidationModuleKt\$passwordValidationModule\$lambda\$6\$\$\$inlined\$singleOf\$default\$1.invoke

Close

0 notes

org.koin.core.error.NoDefinitionFoundException - No definition found for type 'java.lang.Integer'. Check your Modules configuration and add missing type and/or qualifier!

[kz.berekebank.business.password.password_validation.impl.di](#) [SingleOf.kt:225](#) [Fresh issue](#)

Filter

Event type = "Crashes"



Last 7 days

Oct 23 - Oct 29

This issue has 1 crash event affecting 1 user



Gemini has analyzed the stack trace and other metrics, and here's the analysis:

Total events by version

for the last 7 days

1.50.0-CPT-3286-87 (158)

1



Devices

100% Xiaomi

Operating systems

100% Android 11

Device states

0% background

100% Xiaomi →

Events

Event summary

1.50.0-CPT-3286-87 (158)

11

Redmi 9

Oct 28, 2025, 5:06:55 PM



Stack trace

Keys

Logs & Breadcrumbs

Data

Rollouts



TXT



Fatal Exception: org.koin.core.error.InstanceCreationException

Could not create instance for '[Factory: 'kz.berekebank.business.password.password_validation.impl.presentation.screens.validate_password.ValidatePasswordViewModel']'

```
Fatal Exception: org.koin.core.error.InstanceCreationException: Could not create instance for '[Factory: 'kz.berekebank.business.password.
at org.koin.core.instance.InstanceFactory.create(InstanceFactory.kt:56)
at org.koin.core.instance.FactoryInstanceFactory.get(FactoryInstanceFactory.kt:38)
at org.koin.core.registry.InstanceRegistry.resolveInstance$koin_core(InstanceRegistry.kt:132)
```

◆ Gemini has analyzed the stack trace and other metrics, and here's the analysis:

The crash is an `org.koin.core.error.NoDefinitionFoundException` with the message "No definition found for type 'java.lang.Integer'. Check your Modules configuration and add missing type and/or qualifier". This indicates that the Koin dependency injection framework is unable to find a definition for injecting an `Integer` type. This issue is common in applications using dependency injection when a required dependency is not properly declared in the Koin modules. The crash occurs within the `PasswordValidationModuleKt` during the creation of a `PasswordRulesValidator`, which is a dependency for `ValidatePasswordViewModel`.

Potential Causes:

- **Missing Koin Definition:** The most direct cause is that an `Integer` type is required as a dependency by `PasswordRulesValidator` or one of its dependencies, but there is no Koin definition (e.g., `single`, `factory`, `get`) for an `Integer` in `PasswordValidationModuleKt` or any of the loaded Koin modules.
- **Incorrect Koin Scope/Qualifier:** If an `Integer` definition exists but with a specific qualifier or scope, and the consumer is not requesting it with that qualifier or within that scope, Koin will report it as not found.
- **Transitive Dependency:** The `PasswordRulesValidator` itself might not directly require an `Integer`, but one of its internal dependencies does. Koin's inability to resolve this transitive dependency propagates up.
- **Configuration Error:** A typo or incorrect configuration in the Koin module definition, specifically when trying to provide an `Integer` value.
- **Android 11 Specific Behavior (Unlikely for this specific error):** While the crash is reported on a Xiaomi device, and Android 11 is mentioned, a `NoDefinitionFoundException` in Koin is fundamentally an application-level dependency injection configuration issue. It's highly unlikely that Android 11 directly causes Koin to forget or fail to register a definition for a basic type like `Integer`. Android 11 changes typically relate to storage, permissions, background execution, or UI behaviors, not core DI framework operations for basic types. The Xiaomi-specific occurrence might be coincidental or related to a device-specific configuration that exposes this underlying DI issue more frequently, perhaps due to different app lifecycle management or background process termination, but it's not the root cause of the DI problem itself.

Recommendations for Analysis and Debugging:

1. Examine `PasswordValidationModuleKt` :

- Locate the `passwordValidationModule` definition.
- Identify where `PasswordRulesValidator` is defined (likely using `singleOf` or `factoryOf`).
- Inspect the constructor of `PasswordRulesValidator` and any of its dependencies.
- Look for any parameters of type `Int` or `Integer` in these constructors.
- If an `Integer` parameter is found, ensure there's a corresponding Koin definition for it within the `passwordValidationModule` or a module included in `startKoin`.

Example of a missing definition:

```
// In PasswordRulesValidator.kt
class PasswordRulesValidator(private val minLength: Int) {
    // ...
}

// In PasswordValidationModuleKt where the crash occurs
val passwordValidationModule = module {
    singleOf(::PasswordRulesValidator) // Koin doesn't know how to provide 'minLength: Int'
    // Missing: single { 8 } // Or some other Integer value
```

ЛОВУШКА #1: "LLM ЭТО ИИ, САМ РАЗБЕРЁТСЯ", ПРИМЕР

Задача: разобрать краши

- Google интегрировал в firebase crashlytics Gemini
- Доступа к кодовой базе нет
- Решения часто бесполезны
- Фиксы делаются наугад

Ловушка #1: "LLM это ИИ, сам разберётся", ПРИМЕР

Решение:

1. Чёткий контекст
2. Конкретные ожидания
3. Проработанный промпт
4. Проверка результата

```
# Firebase Crashlytics Analysis System

Ты Staff Android Developer, специалист мирового уровня по багам. Ты работаешь над очень важным для тебя проектом. Для доступа к исходному коду ты используешь MCP подключенные к Claude.
За каждый правильно проанализированный баг ты получаешь $5000.

## Цель проекта
Анализ стектрейсов из Firebase Crashlytics для выявления и устранения багов в банковском Android-приложении для юридических лиц. Поддержание метрики crash-free users выше 99%.

## Структура информации о крашах
При добавлении нового крашлога, следует предоставлять:
1. Полный стектрейс
2. Версию приложения
3. Устройство и версию Android
4. Частоту возникновения краша
5. Действия пользователя, которые привели к крашу (если известны)

## Процесс анализа
1. **Классификация краша**:
   - Критичность (блокирующий, критический, некритический)
   - Компонент приложения (UI, сетевой слой, бизнес-логика и т.д.)
   - Тип исключения (NPE, OutOfMemory, IndexOutOfBoundsException и т.д.)
2. **Анализ корневой причины**:
   - Разбор стектрейса строка за строкой
   - Выявление потенциальных проблемных мест в коде
   - Анализ контекста использования приложения в момент краша
   - При анализе кода используй Model Context Protocol (MCP) для доступа к исходному коду

## MCP Стратегии и Fallback**

### Основной подход с MCP:
1. Начинать анализ стектрейса сразу, не делая автоматических попыток чтения файлов без понимания полной структуры проекта
2. Использовать код MCP только после анализа стектрейса и определения конкретных файлов, которые действительно нужны для анализа (с правильными путями)
3. При анализе крашей начинать с выявления ключевых файлов и классов на основе стектрейса, затем делать целенаправленные запросы к MCP
4. Используй команды MCP для просмотра определений классов/методов, упомянутых в стектрейсах
5. Для анализа корневой причины обязательно исследуй через MCP:
   - Классы и методы, упомянутые в верхней части стектрейса
   - Родительские классы и интерфейсы
   - Связанные классы в том же пакете
6. Проактивно предлагай решения на основе глубокого понимания кода, полученного через MCP
7. Указывая в анализе, какие файлы и методы ты проверил через MCP

### **Fallback стратегии когда MCP не работает:**
Если поиск через MCP не дает результатов или возвращает пустые ответы:
1. **Анализируй стектрейс БЕЗ доступа к коду** - часто этого достаточно для понимания проблемы
2. **Используй экспертные знания Android**:
   - AndroidX компоненты имеют известные race conditions
   - Системные ограничения (Doze mode, Батарея) влияют на фоновые задачи
   - Многие android.* краши решаются на уровне приложения без изменения библиотек
3. **Предлагай универсальные решения** для типовых Android проблем
4. **В файле указывай**: "TBD - требует поиска конкретного файла в IDE" вместо "файл не найден"
5. **Создавай решения на основе паттернов** из стектрейса и знаний Android архитектуры

### **Android/Firebase специфика**:
- JobIntentService race conditions характерны для Firebase/GMS библиотек
- AsyncTask + Job Scheduler часто дают IllegalArgumentException
- Firebase Messaging Service может использовать внутренние JobIntentService
- Системные ограничения Android влияют на фоновые задачи
- Многие краши в сторонних библиотеках решаются обработчиками в app коде

При работе с подключенными MCP серверами всегда обращайся к коду напрямую, а не делай предположения о его структуре или реализации.
```


Ловушка #2: "ВЫГЛЯДИТ УБЕДИТЕЛЬНО - ЗНАЧИТ ПРАВДА"

Потеря критического мышления



"Inner Sound 108" Naoto Hattori

Ловушка #2: "ВЫГЛЯДИТ УБЕДИТЕЛЬНО - ЗНАЧИТ ПРАВДА"

Потеря критического мышления

- LLM уверенно выдаёт галлюцинации
- Принимаешь ответ без проверки
- Получаешь проблемы в ci\cd или продакшене



"Inner Sound 108" Naoto Hattori

Ловушка #2: "ВЫГЛЯДИТ УБЕДИТЕЛЬНО - ЗНАЧИТ ПРАВДА"

Совет:

Всегда проверяй результат, относись к LLM как к генератор идей, а не как истину в последней инстанции



"Inner Sound 108" Naoto Hattori

ЛОВУШКА #2: "ВЫГЛЯДИТ УБЕДИТЕЛЬНО - ЗНАЧИТ ПРАВДА", ПРИМЕР

Задача: рефакторинг deprecated методов

- LLM уверенно выдаёт актуальные замены
- Принимаешь ответ без проверки
- Добавляешь импорты и зависимости
- А методов нет...

Ловушка #2: "ВЫГЛЯДИТ УБЕДИТЕЛЬНО - ЗНАЧИТ ПРАВДА", ПРИМЕР

Решение:

Следить за контекстом и иногда заглядывать в документацию.



What weighs more, two pounds of feathers or a pound of bricks?



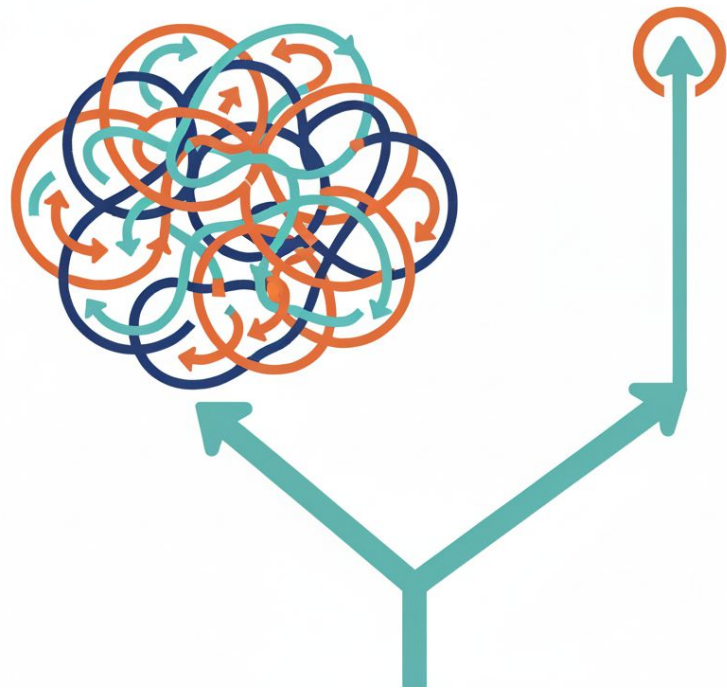
Two pounds of feathers and a pound of bricks both weigh the same amount, which is one pound.



Ловушка #3: "Я уже час выбиваю решение этой проблемы, ещё чуть-чуть..."

Невозвратные затраты

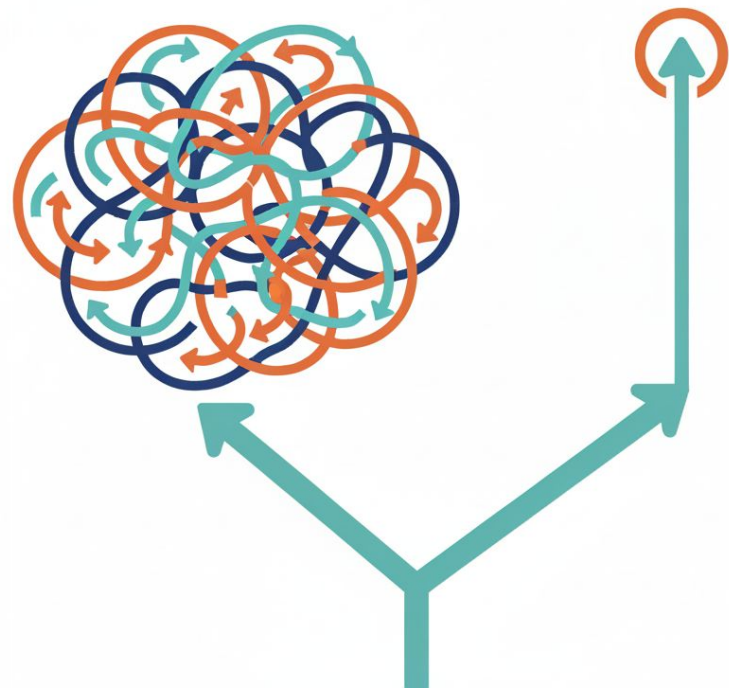
- Итераций больше, а толку меньше
- "Жалко выбрасывать"
- 2 часа с LLM vs 20 минут самостоятельно



Ловушка #3: "Я уже час выбиваю решение этой проблемы, ещё чуть-чуть..."

Совет:

Если после 3-4 итераций результат не приближается к цели — начни заново с нового чата или смени подход



Ловушка #3: "Я УЖЕ ЧАС ВЫБИВАЮ РЕШЕНИЕ ЭТОЙ ПРОБЛЕМЫ, ЕЩЁ ЧУТЬ-ЧУТЬ...", ПРИМЕР

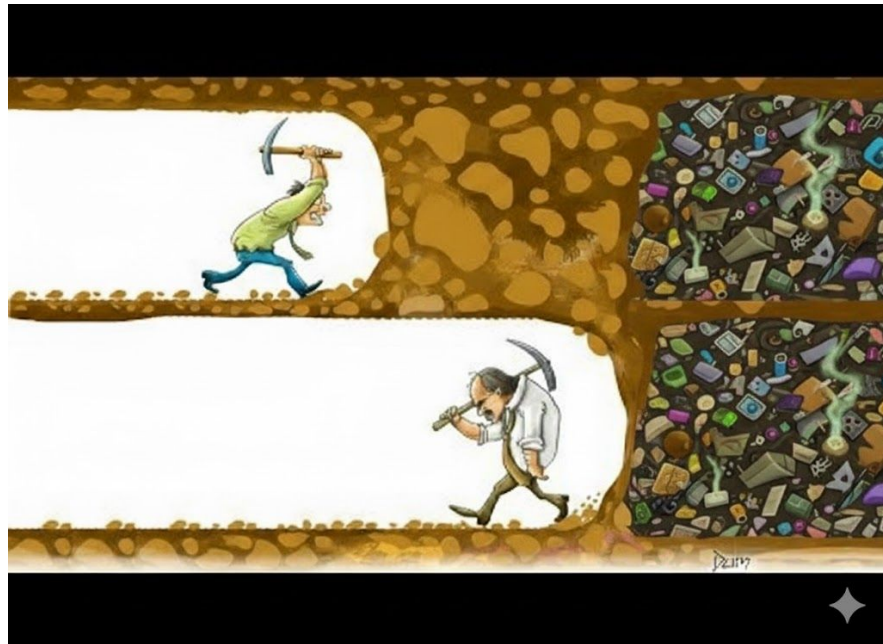
Задача: фикс краша камеры

- проанализировали, выявили причину
- добавили новый модуль, тесты
- Дорабатываем, учитываем edge cases

Ловушка #3: "Я уже час выбиваю решение этой проблемы, ещё чуть-чуть...", ПРИМЕР

Решение:

На второй или третий день понял, что для решения краша, который возник у 5 пользователей, нужен не новый модуль, а простой try-catch в 5 строчек с предложением повторить попытку.



СОВЕТЫ

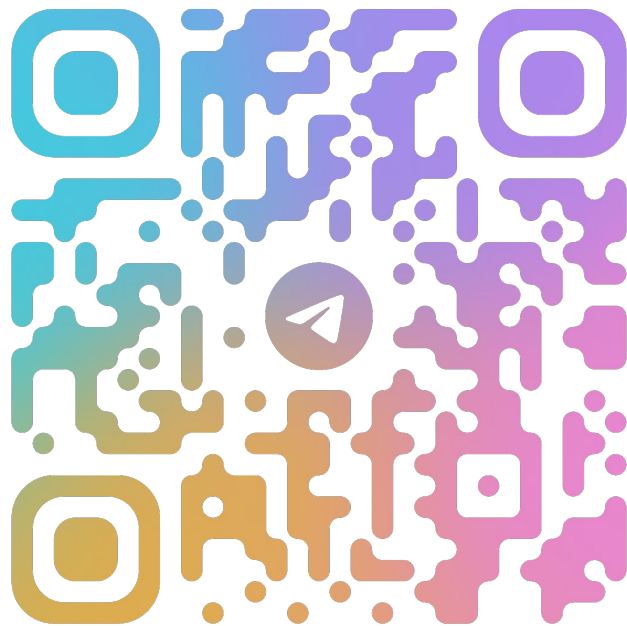
- добавлять контекст
- не переполнять контекст

СОВЕТЫ

- добавлять контекст
- не переполнять контекст
- анализировать результат
- тестировать

СОВЕТЫ

- добавлять контекст
- не переполнять контекст
- анализировать результат
- тестировать
- изучать best practices по работе с LLM
- осознавать разницу между vibe- и meta-coding и использовать их сообразно ситуации



тг каналы про LLM

LLM — НЕ ТВОЙ ДЖУН И НЕ ТВОЙ СЕНЬОР EX MACHINE.

LLM ТВОЙ СТРАННЫЙ НАПАРНИК СО СВОИМИ СИЛЬНЫМИ И СЛАБЫМИ
СТОРОНАМИ. ПРИТРИТЕСЬ — И ВПЕРЁД К ПАРНОМУ
ПРОГРАММИРОВАНИЮ.



БУДУТ ВОПРОСЫ

ОБРАЩАЙТЕСЬ

